



Student Edition

User Guide

From First Install to Running Verilog RTL Simulations in VS Code

R.R.LOGIC SYSTEMS PRIVATE LIMITED

September 2026 · v1.0



Chapter

Table of Contents

1. Introduction

- What is Hydra Sim Studio?
- Who This Guide Is For
- Document Conventions

2. System Requirements

- Supported Platforms
- Hardware Requirements
- Software Prerequisites

3. Installation

- Installing from the VS Code Marketplace
- Verifying the Installation
- First Launch

4. Project Setup & hydra.yaml Configuration

- Creating a Verilog Project
- Understanding hydra.yaml
- Sources Configuration
- Simulation Parameters
- Complete Example

5. Compiling Your Workspace

- Opening the Hierarchy Browser
- Running the Compiler
- Reading Diagnostics
- Error Navigation

6. Running Your First RTL Simulation

- Pre-Flight Checklist
- Launching a Simulation
- The Waveform Window
- Stepping Through Events

7. Waveform Viewer Deep Dive

- Panel Overview
- Navigation Controls
- Zoom & Pan
- Measurement Markers
- Frozen Waveforms
- Radix Selection

8. Editor Decorations & Event Queues

- Colour Coding System
- Inline Annotations
- Queue Tables



9. Hierarchy Browser

- Browsing the Design Tree
- Setting Working Hierarchy
- Per-Instance Waveforms

10. RTL AI Assistant

- What the Assistant Can Do
- Opening the Panel
- Querying Your Design
- Local AI Model

11. Keyboard Shortcuts Reference

12. Troubleshooting

- Common Issues
- Platform Notes
- Getting Help

13. Comparing Free Verilog Simulators

- Tools Covered
- Feature Matrix
- Dimension-by-Dimension Analysis
- Choosing the Right Tool



Chapter 1

Introduction

Hydra Sim Studio brings event-driven Verilog 2001 simulation directly into VS Code — no separate EDA tool needed.

1.1 What is Hydra Sim Studio?

Hydra Sim Studio is a free VS Code extension developed by R.R.Logic Systems Private Limited. It embeds a Verilog 2-state simulator engine inside your editor, letting you compile, simulate, step through, and visualise RTL behaviour without switching to a separate EDA application. All interaction — from compilation diagnostics to live waveform rendering — happens within familiar VS Code panels and keyboard shortcuts.

Key capabilities include real-time waveform display, colour-coded event-queue highlighting in the source editor, an interactive hierarchy browser, and an AI-powered RTL assistant that can answer design questions about your specific Verilog project.

1.2 Who This Guide Is For

This guide targets students and early-career engineers who are new to Verilog RTL simulation. It assumes you are comfortable with the VS Code interface and have a basic understanding of digital logic; no prior EDA or simulation experience is required.

1.3 What You Will Learn

- Install and activate the Hydra Sim Studio extension
- Configure a project with a *hydra.yaml* file
- Compile a Verilog workspace and interpret error diagnostics
- Run a simulation and navigate the waveform viewer
- Step through the Verilog event scheduler one queue at a time
- Explore design hierarchy and scope waveforms per instance
- Query your RTL with the built-in AI assistant

1.4 Document Conventions

Convention	Meaning
Ctrl+Shift+F9 (keyboard shortcut)	Press keys simultaneously
<i>hydra.yaml</i> (italic)	File name or configuration field
<code>working_hierarchy: tb_top</code> (code)	YAML / code literal — enter exactly as shown



Convention	Meaning
Bold text	UI element label (button, menu item, panel name)



Chapter 2

System Requirements

Review these requirements before installing the extension to avoid compatibility issues.

2.1 Supported Platforms

■ **NOTE:** Hydra Sim Studio currently runs on **Linux** and **Windows Subsystem for Linux (WSL)**. Native Windows operation (outside WSL) is not yet supported. macOS support has not been announced.

Platform	Supported	Notes
Linux (any distro)	YES	Full support — recommended environment
Windows via WSL 2	YES	Launch VS Code inside WSL; see Chapter 3
Windows (native)	NO	Native Windows not currently supported
macOS	NOT ANNOUNCED	Check marketplace page for updates

2.2 Hardware Requirements

Minimum (core extension only)

- Any modern 64-bit CPU
- 4 GB RAM
- 500 MB free disk space
- VS Code 1.74 or later

Recommended (with local AI model)

- NVIDIA GPU with **8 GB VRAM** (required for local AI model download)
- **8 GB RAM** system memory
- **20 GB free disk space** for model weights

■ **NOTE:** The built-in structural/simulation assistant works without a GPU. The local AI model download is optional and only needed for enhanced natural-language responses.

2.3 Software Prerequisites

Software	Minimum Version	Where to Get It
Visual Studio Code	1.74+	https://code.visualstudio.com
Git (optional)	Any	For version-controlling your Verilog files
WSL 2 (Windows only)	WSL 2	Windows Features or <code>wsl.exe --install</code>



■ **IMPORTANT:** Do **not** install any separate Verilog simulator (Icarus, Verilator, ModelSim, etc.). Hydra Sim Studio ships its own simulation engine; a conflicting tool on your PATH will not affect it, but it is also not required.

Chapter 3

Installation

Installing Hydra Sim Studio takes under two minutes from the VS Code marketplace.

3.1 Installing from the VS Code Marketplace

Method A — Extensions Panel (Recommended)

1. Open VS Code.
2. Click the **Extensions** icon in the Activity Bar (or press **Ctrl+Shift+X**).
3. In the search box type: **Hydra Sim Studio**
4. Locate the entry published by **RRLOGICSYSTEMSPRIVATELIMITED**.
5. Click the blue **Install** button.
6. Wait for the status bar to show *Installing...* then *Installed*.

Method B — Quick Open Command

Press **Ctrl+P** to open Quick Open, paste the following line, and press Enter:

Quick Open — paste this line

```
ext install RRLOGICSYSTEMSPRIVATELIMITED.Hydra-Sim-Studio
```

Method C — Command Palette

Press **Ctrl+Shift+P**, type *Extensions: Install Extensions*, search for *Hydra Sim Studio*, and click Install.

3.2 Verifying the Installation

After installation a new icon appears in the VS Code Activity Bar on the left — this is the **Hydra Hierarchy Browser**. You can also confirm by opening **Extensions** and checking that Hydra Sim Studio shows a green ✓ badge.

1. Open the Command Palette (**Ctrl+Shift+P**).
2. Type *Hydra* — you should see at least six commands listed (Run Simulation, Step Forward, Step Backward, Exit Replay, Open RTL Assistant, Compile Workspace).
3. If no Hydra commands appear, reload the window with **Ctrl+Shift+P** → **Developer: Reload Window**.

3.3 WSL Setup (Windows Users)

On Windows, VS Code must be opened inside the WSL environment — not natively. Follow these steps:

1. Install WSL 2 by opening PowerShell as Administrator and running: `wsl --install` then restart your computer.
2. Open your WSL terminal (Ubuntu or similar).



3. Install VS Code's **WSL extension** from the marketplace if prompted.
4. From inside the WSL terminal run: `code .` — VS Code will open with a WSL remote connection indicator in the bottom-left corner.
5. Install Hydra Sim Studio from within this WSL-connected VS Code window.

■ **NOTE:** If VS Code's status bar bottom-left shows **WSL: Ubuntu** (or your distro name), you are correctly connected. Extensions installed here run in Linux — the correct environment for Hydra Sim Studio.



Chapter 4

Project Setup & hydra.yaml Configuration

Every Hydra Sim Studio project is driven by a single YAML configuration file. Nothing compiles or simulates without it.

4.1 Creating a Verilog Project

Organise your work in a dedicated folder. A minimal project contains at least one Verilog source file and (once created) a *hydra.yaml*. A recommended structure is:

Recommended project layout

```
my_project/
├── hydra.yaml ← configuration file (root)
├── tb_counter.v ← testbench (top-level)
├── rtl/
└── counter.v ← design-under-test
```

Open this folder in VS Code: **File** → **Open Folder...** and select *my_project/*. All paths in *hydra.yaml* are relative to the folder containing the file itself.

4.2 Creating hydra.yaml

When you first compile (see Chapter 5), Hydra Sim Studio automatically creates a default *hydra.yaml* next to your open Verilog file if none exists. Alternatively, create it manually at the workspace root. The file uses standard YAML syntax.

■ **IMPORTANT: Nothing runs without hydra.yaml.** Compilation, simulation, and the RTL Assistant all require a valid configuration file. If commands appear greyed-out or do nothing, the first thing to check is whether *hydra.yaml* exists and is correctly formed.

4.3 Configuration Fields Reference

Field	Required	Default	Description
sources	YES	—	List of .v / .sv files or directories to include in the workspace con
working_hierarchy	YES	—	Top-level module/instance name; roots the Hierarchy Browser a
start_time	No	0	Simulation viewport start time
time_unit	No	ns	Time unit: fs, ps, ns, us, ms, or s
max_sim_time	No	1000	Simulation stop time (in chosen time_unit)
max_sim_cycles	No	5000	Hard cycle-count limit to prevent runaway sims



Field	Required	Default	Description
waveform_window_size	No	-1	Initial waveform viewport width; -1 = full timeline
max_frozen_windows	No	5	Number of frozen (previous-run) waveform panels to retain

4.4 Source Path Patterns

The *sources* list accepts several entry formats, allowing precise control over which files are included:

Entry Format	Effect
./counter.v	Include a single specific file
./rtl/	Include all .v files directly in the rtl/ directory (no recursion)
./rtl/*	Include rtl/ files plus one level of subdirectories
"!./rtl/scratch.v"	Exclude a specific file (note the leading !)
"!./rtl/old/"	Exclude an entire directory

■ **NOTE:** Exclusion entries (! prefix) must be quoted in YAML because the exclamation mark is a reserved character.

4.5 Complete hydra.yaml Example

hydra.yaml — complete annotated example

```
# hydra.yaml – complete project configuration

sources:
- ./tb_counter.v # testbench at project root
- ./rtl/ # all files in rtl/ directory
- "!./rtl/scratch.v" # exclude scratch file

working_hierarchy: "tb_counter" # top-level testbench instance

# Simulation parameters
start_time: 0
time_unit: ns
max_sim_time: 1000
max_sim_cycles: 5000

# Waveform viewer
waveform_window_size: -1 # -1 = show full timeline
max_frozen_windows: 5
```

4.6 hydra.yaml Search Order

When you open a Verilog file, Hydra Sim Studio searches for the configuration file in this order:

1. The **workspace root** (top-level folder open in VS Code)
2. The **directory of the currently open .v file**



3. A newly generated default file created next to the Verilog file

Always keep one *hydra.yaml* at the workspace root for multi-file projects to ensure a consistent configuration regardless of which file has editor focus.



Chapter 5

Compiling Your Workspace

Compilation resolves your source files, checks syntax, and populates the Hierarchy Browser.

5.1 Opening the Hierarchy Browser

Click the **Hydra icon** (circuit/wave symbol) in the VS Code Activity Bar on the left. This opens the **Hierarchy** side panel, your main control point for compilation and design exploration.

5.2 Running the Compiler

You can trigger compilation in three equivalent ways:

Method	Action
Keyboard shortcut	Press <code>Ctrl+Shift+F8</code> with the editor in focus
Command Palette	<code>Ctrl+Shift+P</code> → Hydra: Compile Workspace
Hierarchy Browser button	Click the <code>Compile</code> (refresh) icon in the Hierarchy panel title bar

Compilation runs in the background. A progress indicator appears in the VS Code status bar. When complete, the Hierarchy Browser populates with your design tree.

5.3 Reading Diagnostics

Any syntax errors or warnings are reported directly in VS Code's built-in **Problems** panel (`Ctrl+Shift+M`). Each diagnostic entry shows:

- The source file name and line number (clickable — jumps to the error location)
- Error severity: **Error** (blocks simulation) or **Warning** (advisory only)
- A description of the problem

■ **NOTE:** Compilation re-runs automatically in the background whenever you save a `.v` file, keeping the Problems panel continuously up-to-date as you edit.

5.4 Common Compilation Errors

Symptom	Likely Cause	Fix
No files appear in Hierarchy Browser	<code>hydra.yaml</code> missing or sources list empty	Create / edit <code>hydra.yaml</code> — see Chapter 4
'Undefined module' error	A <code>.v</code> file is not listed in sources	Add the file or its directory to sources:
'Unexpected token' error	Syntax mistake in Verilog	Click the error link to jump to the offending line
Hierarchy Browser shows wrong topology	<code>working_hierarchy</code> does not match module name	Update <code>working_hierarchy</code> in <code>hydra.yaml</code>

Chapter 6

Running Your First RTL Simulation

Once your workspace compiles cleanly, you are three keystrokes away from a live waveform.

6.1 Pre-Flight Checklist

Confirm all four items before launching the simulation:

- **hydra.yaml present** — at workspace root or alongside your top-level file
- **sources resolved** — at least one .v / .sv file included
- **working_hierarchy set** — must match the name of your testbench module
- **Compilation passes** — no Errors in the Problems panel (warnings are okay)

6.2 Launching a Simulation

1. Open your top-level Verilog file (the testbench) in the editor.
2. Press **Ctrl+Shift+F9** (or **Ctrl+Shift+P** → **Hydra: Run Simulation**).
3. The simulation engine runs; a progress bar appears briefly in the status bar.
4. The **Hydra Live Waveform** panel opens in a new VS Code window automatically.
5. Your editor source lines are immediately colour-highlighted (event queues — see Chapter 8).

■ **NOTE:** Simulation key bindings require the editor text area to have focus. Click inside the editor pane first if the shortcut does not respond.

6.3 What Happens Behind the Scenes

When you press Run Simulation, Hydra Sim Studio performs these steps automatically:

1. Re-compiles any source files changed since the last build
2. Elaborates the design starting from *working_hierarchy*
3. Executes the simulation up to *max_sim_time* or *max_sim_cycles*
4. Captures all signal transitions into a simulation trace
5. Renders the full waveform timeline in the Waveform panel
6. Sets the editor to replay mode at simulation time 0

6.4 Stepping Through the Simulation

In replay mode you navigate the simulation event-by-event rather than watching it play in real time. Use these shortcuts (editor must have focus):



Action	Shortcut	Description
Step Forward	Ctrl+Shift+F10	Advance to the next event queue step
Step Backward	Ctrl+Shift+F11	Rewind to the previous event queue step
Exit Replay	Ctrl+Shift+F12	Clear all decorations, close waveform, return to normal editing

As you step, the editor highlights the active source line(s) and updates inline annotations showing current signal values and simulation timestamps. The Waveform panel cursor also moves in sync with your position in the timeline.

6.5 A Worked Example

The following minimal testbench exercises a 4-bit counter. Create these two files and the `hydra.yaml` from Section 4.5 to follow along:

`counter.v`

```
// counter.v – 4-bit up-counter
module counter(
  input wire clk,
  input wire rst,
  output reg [3:0] count
);
always @(posedge clk or posedge rst) begin
  if (rst)
    count <= 4'b0000;
  else
    count <= count + 1;
end
endmodule
```

`tb_counter.v`

```
// tb_counter.v – testbench
module tb_counter;
  reg clk = 0;
  reg rst = 1;
  wire [3:0] count;

  counter dut(.clk(clk), .rst(rst), .count(count));

  always #5 clk = ~clk; // 100 MHz clock

  initial begin
    #20 rst = 0; // release reset
    #100 $finish; // end after 100 ns
  end

  initial $monitor($time, " rst=%b count=%d", rst, count);
endmodule
```

With these files saved and `hydra.yaml` configured, press **Ctrl+Shift+F8** to compile, verify there are no errors, then press **Ctrl+Shift+F9** to run. You will see the waveform panel open with `clk`, `rst`, and `count` signals.

Chapter 7

Waveform Viewer Deep Dive

The Hydra Live Waveform panel is a full-featured signal viewer that stays synchronised with your editor as you step through simulation.

7.1 Panel Overview

The waveform panel opens in a separate VS Code window when a simulation completes. It contains three main areas:

- **Signal list (left panel)** — lists each monitored signal with its current value, source module, and trigger information
- **Waveform canvas (centre)** — renders signal transitions as a timeline; the time cursor moves as you step in the editor
- **Queue table (right/bottom panel)** — lists the five event queues; clicking an entry jumps the editor to the corresponding source line

■ **NOTE:** The waveform panel keeps its state even when hidden. Switching back to it does not require a re-run — the last simulation data is preserved.

7.2 Navigation Controls

Control	Action
■ / ■ arrows	Step backward / forward one event queue step (mirrors editor shortcuts)
Transition jump buttons	Jump to the previous / next value change on the selected signal
Edge jump buttons	Jump to previous / next clock edge (single-bit signals only)
Ctrl+–	Zoom out
Ctrl+=	Zoom in
Ctrl+0	Fit — reset zoom to show the entire simulation
Start button	Re-fetch the waveform window from a new time offset without re-running
m key	Drop a measurement marker at the current cursor position
n or Escape	Clear the measurement marker

7.3 Zoom & Pan

The waveform canvas supports two interaction modes for exploring long simulations:

- **Drag to pan** — click and drag the canvas horizontally to scroll through time



- **Ctrl+scroll to zoom** — hold Ctrl and use the mouse wheel to zoom in or out centred on the pointer position

Use **Ctrl+0** (Fit) to return to the full-timeline view at any point.

7.4 Measurement Markers

Press **m** to drop a time marker at the cursor. Select a signal and place two markers to enable the transition-jump buttons, which let you quickly skip to the next time the signal changes value. Press **n** or **Escape** to clear markers.

7.5 Frozen Waveforms

Each time you run a new simulation, the previous waveform panel is **frozen** — its title changes to *Hydra Waveform (Frozen)* — while the new run opens a fresh live panel. This lets you compare two simulation runs side-by-side without needing a third-party diff tool.

The number of frozen panels retained is controlled by *max_frozen_windows* in *hydra.yaml* (default: 5). Older frozen panels are automatically closed when the limit is exceeded.

7.6 Radix Selection for Bus Signals

For multi-bit (bus) signals, a **Radix** selector appears in the signal list. Choose from:

- **Decimal** — unsigned base-10
- **Hex** — base-16 (prefixed 0x)
- **Binary** — bit-by-bit

The radix setting applies per signal and persists until you change it or close the panel.

Chapter 8

Editor Decorations & Event Queues

Hydra Sim Studio colour-codes your source lines by their position in the Verilog event scheduler, making it possible to see exactly how the simulator processes each timestep.

8.1 The Verilog Event Scheduler

Verilog simulation is fundamentally event-driven. Within a single simulation timestep the simulator processes events in a strict priority order across five queues. Understanding this order is key to debugging race conditions, glitches, and non-blocking assignment (NBA) update timing.

8.2 Colour Coding System

While in replay mode, active source lines are highlighted according to which event queue they currently belong to:

	Queue	Event Type	Meaning
	Active	Active Queue	Currently executing statements in this timestep
	Inactive	Inactive Queue	Events deferred to #0 delay within the same timestep
	NBA	NBA Queue	Non-blocking assignment (<code><=></code>) register updates
	Monitor	Monitor Queue	<code>\$monitor</code> / <code>\$display</code> callbacks
	Future	Future Queue	Events scheduled at a later simulation timestep

Multiple lines can be highlighted simultaneously if events belonging to different queues are active at the same simulation step. A floating **Legend** overlay in the editor reminds you of the colour mapping during active replay sessions.

8.3 Inline Annotations

In addition to background colour, the extension inserts lightweight inline annotations as greyed-out comments after relevant lines:

Annotation	Location	Content
Simulation Time	Line below module header	// Simulation Time = 45 (in chosen time_unit)
Signal Value	Next to a reg/wire declaration	// Value=3
Queue Timestamp	Next to a queued statement	// When=50

These annotations update live with every step forward or backward, so you always see the exact state of the design at the current simulation time.



8.4 Queue Tables in the Waveform Panel

The **Queues Table** in the waveform panel lists all events currently in each queue. Clicking any entry in the table immediately jumps the editor to the source line responsible for that event — extremely useful for tracking down which always-block or assignment is driving an unexpected value.

Chapter 9

Hierarchy Browser

The Hierarchy Browser gives you a structural view of your Verilog design and provides direct control over waveform scoping.

9.1 Browsing the Design Tree

The Hierarchy Browser is accessed by clicking the **Hydra icon** in the Activity Bar. It displays a collapsible tree rooted at the module specified in *working_hierarchy*. Each node in the tree shows:

- **Instance name** — the name used at the instantiation site (full font)
- **Module name** — the module definition name (dimmed / smaller font)

Clicking any node opens the corresponding source file at the module's instantiation point, making it easy to navigate large designs without manually searching files.

9.2 Title Bar Controls

Icon / Button	Action
Search Instance (magnifier)	Filter the tree by instance or module name
Refresh Hierarchy (cycle)	Re-read the compiled design; use after editing sources
Go to Parent (up-arrow)	Move the tree root up one level
Collapse All	Collapse all expanded nodes

9.3 Right-Click Context Menu

Menu Item	Effect
Set as Current Working Hierarchy	Re-roots the browser and elaboration at this instance
Add to Waveform	Includes this instance's signals in the waveform panel
Remove from Waveform	Excludes this instance's signals from the waveform panel

9.4 Per-Instance Waveform Scoping

In large designs with many instances of the same module, per-instance scoping is invaluable. Right-click any instance and choose **Add to Waveform** to add only that instance's signals to the waveform panel. Use **Set as Current Working Hierarchy** to zoom the browser and elaboration into that subtree. This lets you compare the behaviour of two instances of the same module side-by-side.

Chapter 10

RTL AI Assistant

The built-in AI assistant answers structural and behavioural questions about your compiled Verilog design in natural language.

10.1 What the Assistant Can Do

- Answer questions about signal connectivity, module hierarchy, and port lists
- Explain what a particular always-block or assign statement does
- Describe the timing behaviour observed in a simulation run
- Report compilation errors as clickable file/line links
- Validate design intent against protocol datasheets (with enhanced local model)

10.2 Opening the RTL Assistant

Press **Ctrl+Shift+J** to toggle the RTL Assistant chat panel. Alternatively, use the Command Palette: **Hydra: Open RTL Assistant**.

The assistant panel opens automatically when a Verilog file becomes the active editor. If your workspace has not yet been compiled when you open the panel, the assistant will display any pending compilation errors and prompt you to compile first.

■ **NOTE:** A compilation with *hydra.yaml* configured is required before the assistant can answer design-specific questions. Generic Verilog questions work at any time.

10.3 Querying Your Design

Type questions in plain English. Example queries:

- "What signals does the counter module expose?"
- "Which always-block drives the count register?"
- "At simulation time 45 ns what is the value of rst?"
- "Why does count not increment on the first clock edge?"

The assistant replies with explanations, and where relevant, clickable links to the source file and line number containing the answer.

10.4 Local AI Model (Optional)

On systems with an NVIDIA GPU and at least 8 GB VRAM, you can download a local AI model for enhanced natural-language responses. A one-click download button appears inside the assistant panel when eligible hardware is detected. The model weighs approximately 20 GB and is stored locally; all inference runs on your machine without sending data to external servers.



■ **IMPORTANT:** The local model download requires **20 GB free disk space** and **8 GB VRAM**. If your hardware does not meet these requirements, the built-in structural assistant continues to function normally — no download is necessary for core functionality.



Chapter 11

Keyboard Shortcuts Reference

All Hydra Sim Studio commands are accessible via the Command Palette. Default shortcuts are listed below; all bindings can be customised in File → Preferences → Keyboard Shortcuts.

11.1 Core Simulation Shortcuts

Command	Default Shortcut	Notes
Hydra: Compile Workspace	Ctrl+Shift+F8	Editor must have focus
Hydra: Run Simulation	Ctrl+Shift+F9	Editor must have focus
Hydra: Step Forward	Ctrl+Shift+F10	Editor must have focus
Hydra: Step Backward	Ctrl+Shift+F11	Editor must have focus
Hydra: Exit Replay	Ctrl+Shift+F12	Clears all decorations
Hydra: Open RTL Assistant	Ctrl+Shift+J	Toggle chat panel

11.2 Waveform Panel Shortcuts

Control	Shortcut / Action
Zoom in	Ctrl+= (inside waveform panel)
Zoom out	Ctrl+- (inside waveform panel)
Fit to timeline	Ctrl+0
Drop marker	m key
Clear marker	n key or Escape
Pan	Click and drag canvas
Zoom around pointer	Ctrl + mouse scroll wheel

11.3 Customising Shortcuts

1. Open **File** → **Preferences** → **Keyboard Shortcuts** (or press **Ctrl+K Ctrl+S**).
2. Search for *Hydra* to filter all extension commands.
3. Click the pencil icon next to any command to assign a new key binding.
4. Press the desired key combination and confirm with Enter.

Chapter 12

Troubleshooting

Solutions to common issues encountered when setting up or running Hydra Sim Studio.

12.1 Common Issues & Solutions

■ Simulation shortcut does nothing

The editor text area must have focus. Click inside the code editor pane first, then press the shortcut. The waveform panel and other VS Code panels steal focus.

■ Hydra commands missing from Command Palette

The extension may not have activated. Open any .v file — the extension activates when a Verilog file is open. If commands still do not appear, run Developer: Reload Window from the Command Palette.

■ hydra.yaml not found

Ensure the file exists at the workspace root (the topmost folder open in VS Code). Check the file is named exactly hydra.yaml (lowercase, no extra extensions). Hydra Sim Studio will auto-generate one if it cannot find it.

■ Hierarchy Browser is empty

The workspace has not been compiled yet, or compilation failed. Check the Problems panel for errors. Verify working_hierarchy matches a module name actually defined in your sources.

■ Waveform panel does not open

The simulation ran but produced no events. Verify your testbench calls \$finish or that max_sim_time / max_sim_cycles are large enough. Also check that the working_hierarchy instance is your testbench, not the DUT.

■ Simulation ends immediately at time 0

max_sim_time or max_sim_cycles is set too low. Increase both values in hydra.yaml. Also verify the testbench has at least one initial block with a time-advancing statement.

■ 'Unsupported construct' compile error

Hydra Sim Studio supports Verilog 2001 only. Constructs like forever, repeat, casex, generate for, and SystemVerilog syntax are not yet supported. Simplify the relevant code or use equivalent Verilog 2001 constructs.

■ Extension does not work on Windows

Native Windows (outside WSL) is not currently supported. Install WSL 2, then open VS Code from inside the WSL terminal using code . See Chapter 3 for step-by-step WSL setup instructions.



■ AI Assistant gives wrong answers

Compile first — Ctrl+Shift+F8 — before querying the assistant. Without a compilation the assistant has no knowledge of your specific design. Check for error links in the assistant response.

12.2 Platform Notes

- **Linux keyboard conflicts:** Function-key shortcuts (F8–F12) are used intentionally to avoid conflicts with Ctrl+Alt combinations that are reserved by many Linux desktop environments (GNOME, KDE, etc.).
- **WSL display:** If VS Code windows do not appear when running inside WSL, ensure your WSL environment has access to a display server (WSLg on Windows 11 provides this automatically).
- **GPU availability in WSL:** NVIDIA GPU passthrough to WSL 2 requires Windows 11 and the WSL-CUDA driver. Without it, the local AI model download button will not appear, but all other features work normally.

12.3 Getting Help

If you encounter an issue not covered here, use the following resources:

- **VS Code Marketplace page** — check the extension page for updated documentation and release notes: marketplace.visualstudio.com
- **Command Palette diagnostics** — run Hydra: Compile Workspace and read the Problems panel carefully; most issues surface as compilation diagnostics.
- **VS Code Output panel** — open **View** → **Output** and select the *Hydra Sim Studio* channel from the dropdown to see raw extension logs.
- **Reload Window** — many activation issues are resolved by pressing **Ctrl+Shift+P** → **Developer: Reload Window**.

Chapter 13

Comparing Free Verilog Simulators

How does Hydra Sim Studio fit alongside the other freely available Verilog simulation tools? This chapter maps the major options across the dimensions that matter most to students and early-career engineers.

13.1 Tools Covered

The table below covers the five most widely used free Verilog simulation environments available in 2026:

Hydra Sim Studio

VS Code extension by R.R.Logic Systems. Embeds a 2-state Verilog 2001 simulator directly in the editor with live waveforms, event-queue stepping, hierarchy browser, and an AI RTL assistant.

Icarus Verilog + GTKWave

The most widely used open-source Verilog simulator (GPL2+). Command-line compiler; pairs with the GTKWave waveform viewer for VCD/FST file display. Supports Verilog 1995/2001/2005 and limited SystemVerilog.

Verilator

GPL3 high-speed simulator that compiles Verilog/SystemVerilog to multithreaded C++. Targets verification engineers writing C++ or SystemC testbenches; 10–100× faster than interpreted simulators.

EDA Playground

Browser-based IDE hosting Icarus Verilog 12, Verilator v5, and GHDL as free back-ends alongside commercial options (Xcelium, Questa, VCS). Includes EPWave waveform viewer and real-time code sharing.

Questa Lite (Intel Quartus Prime Lite)

GUI-driven simulator bundled free with Intel Quartus Prime Lite Edition. Supports Verilog 2001/2005 and limited SystemVerilog; restricted to small designs and non-commercial use.

13.2 Feature Matrix

Feature	Hydra Sim Studio	Icarus + GTKWave	Verilator	EDA Playground	Questa Lite
Cost	Free	Free (GPL2+)	Free (GPL3)	Free tier	Free (non-commercial)
Verilog 2001 support	✓	✓	✓	✓	✓
Verilog 2005 support	✗	✓	✓	✓	✓
SystemVerilog support	✗	■	✓	✓	■
Platform	Linux / WSL	Linux Win / Mac	Linux Win / Mac	Browser (any OS)	Linux / Win



Feature	Hydra Sim Studio	Icarus + GTKWave	Verilator	EDA Playground	Questa Lite
Local installation needed	✓	✓	✓	✗	✓
VS Code integration	Native extension	via ext. (manual)	via ext. (manual)	✗	✗
Built-in waveform viewer	✓	✗	✗	EPWave	✓
External viewer required	✗	GTKWave	GTKWave or other	✗	✗
Event-queue step debugger	✓	✗	✗	✗	✗
Inline signal annotations	✓	✗	✗	✗	✗
Design hierarchy browser	✓	✗	✗	✗	✓
AI RTL assistant	✓	✗	✗	✗	✗
Simulation speed	Moderate	Moderate	Very fast (C++ compile)	Moderate	Moderate
Testbench language	Verilog	Verilog / SV	C++ / SystemC	Verilog / SV	Verilog / SV
\$display / \$monitor	✓	✓	✓	✓	✓
\$dumpfile / VCD output	✗	✓	✓	✓	✓
Frozen waveform compare	✓	✗	✗	✗	✗
Real-time code sharing	✗	✗	✗	✓	✗
Suitable for beginners	✓	■	✗	✓	■
Suitable for production	✗	✓	✓	■	■

✓ = Full support

■ = Partial / limited

✗ = Not supported

Teal column = Hydra Sim Studio

13.3 Dimension-by-Dimension Analysis

■ Ease of Setup for Students

Hydra Sim Studio and EDA Playground are the easiest entry points. Hydra requires VS Code plus a *hydra.yaml* file; EDA Playground needs only a browser and a free account — nothing to install. Icarus Verilog demands command-line confidence (compiling with *iverilog*, running with *vvp*, and launching GTKWave separately as a third step). Verilator has the steepest learning curve: testbenches are written in C++ and the Verilog must be fully synthesizable; it is not suitable for students without C++ experience. Questa Lite requires downloading and installing the full Quartus Prime suite (~20 GB) before any simulation is possible.

■ Verilog / SystemVerilog Coverage

Icarus Verilog 12 and Verilator offer the broadest standard coverage: Verilog 1995/2001/2005 and substantial SystemVerilog 2012/2017/2023 subsets. EDA Playground inherits whatever the selected back-end supports. Hydra Sim Studio targets Verilog 2001 only — the version taught in most

undergraduate curricula — and explicitly excludes several rarely-taught constructs (*forever*, *casex*, *generate for*, etc.). Questa Lite supports Verilog 2001/2005 and a limited SystemVerilog subset for non-FPGA designs.

■ Waveform Visualisation

Hydra Sim Studio's built-in waveform panel is unique in opening automatically and staying synchronised with the editor cursor as you step — no separate window management is required. GTKWave is a powerful standalone viewer that reads VCD/FST files produced by Icarus or Verilator, but loading a new simulation requires manually reopening or refreshing the file. EDA Playground's EPWave is browser-based and reasonably capable. Questa Lite includes an integrated waveform viewer comparable to Hydra's but within a heavyweight IDE. Verilator alone produces no waveform; a separate VCD dumping library or GTKWave is needed.

■ Debugging & Event Visibility

This is Hydra Sim Studio's strongest differentiator. No other free tool exposes the Verilog event scheduler (Active / Inactive / NBA / Monitor / Future queues) as colour-coded editor highlights. Students can literally see which line of code is in which queue at each timestep — invaluable for understanding non-blocking assignments and race conditions. Icarus Verilog offers `$dumpfile` / `$dumpvars` for post-simulation waveform analysis, but no interactive stepping in the source editor. Questa Lite provides source-level stepping but without queue-level visibility.

■ AI Assistance

Hydra Sim Studio is the only free simulator with a built-in AI RTL assistant that is aware of your compiled design hierarchy, signal list, and simulation results. All other tools rely on external AI tools (ChatGPT, GitHub Copilot, etc.) that have no knowledge of your specific design.

■ Performance & Scale

Verilator is by far the fastest, compiling designs to native C++ and achieving 10–100× the throughput of interpreted simulators. It is the tool of choice for large SoC-level verification with millions of clock cycles. Icarus Verilog is well-suited for module-level testbenches up to tens of thousands of cycles. Hydra Sim Studio is optimised for educational simulations; the `max_sim_cycles` limit (default 5000) keeps runtimes short and the UI responsive. EDA Playground and Questa Lite are comparable to Icarus in throughput for typical student designs.

■ Collaboration & Portability

EDA Playground is unique in supporting real-time collaboration and instant code sharing via a URL — no local environment needed on either side. This makes it the best choice for pair programming or sharing a failing testbench with an instructor. Hydra Sim Studio's projects are portable as long as both parties have VS Code and the extension installed; `hydra.yaml` travels with the source files.

13.4 Choosing the Right Tool



Scenario	Recommended Tool
First-ever Verilog simulation — zero prior EDA experience	Hydra Sim Studio or EDA Playground
Understanding non-blocking assignments and event scheduling	Hydra Sim Studio (event-queue visualiser is unique)
Verilog 2005 or SystemVerilog testbench required	Icarus Verilog + GTKWave or EDA Playground
Large-scale simulation (>1M cycles) or C++ co-simulation	Verilator
No local installation — browser-only access	EDA Playground
FPGA-targeted design with Quartus toolchain	Questa Lite (bundled with Quartus Prime Lite)
Sharing a failing testbench with an instructor instantly	EDA Playground (real-time collaboration + URL sharing)
AI-assisted design review within the simulation environment	Hydra Sim Studio (only free tool with built-in AI assistant)
Checking Verilog syntax quickly without full simulation	Icarus Verilog (iverilog -t null for fast syntax check)

■ **NOTE:** Hydra Sim Studio and the other tools are **complementary, not mutually exclusive**. A common student workflow is to prototype and debug in Hydra Sim Studio (fast feedback, visual queue stepping), then validate the same design in Icarus Verilog to confirm behaviour under a broader Verilog standard, and finally synthesise with Quartus or Vivado for FPGA deployment.